Experiment No. 10

Student Name: Ojaswini Thote

PRN: 24070521048

Experiment Title

Load and Display JSON Data using fetch() and jQuery $.getJSON(); Render Results Dynamically in a Table

Software / Tools Required

- Visual Studio Code
- Web browser (Google Chrome or Microsoft Edge)
- HTML5 and CSS
- JavaScript (ES6)
- jQuery 3.7.1
- JSON data files

Practical — Student Details using fetch()

Aim

To load student records from a JSON file using the JavaScript fetch() API, parse the response, and dynamically display student ID, name, department, and marks in an HTML table.

Files Used

- index.html — interface and fetch() logic
- students.json — source student records

Program Code

index.html

```
<!DOCTYPE html>
<html>
    <head>
        <title>Load JSON using Fetch</title>
        <style>
            table{
                width: 70%;
                border-collapse: collapse;
                margin-top: 20px;
            }

            th,td{
                border: 1px solid black;
                padding: 10px;
                text-align: left;
            }

            th{
                background-color: #f2f2f2;
            }
        </style>

    </head>

    <body>
```

```
        <h2>Student Details using Fetch()</h2>
        <button onclick="loadStudents()">Load Students</button>

        <table id="studentTable">
            <thead>
                <tr>
                    <th>ID</th>
                    <th>Name</th>
                    <th>Department</th>
                    <th>Marks</th>
                </tr>
            </thead>
            <tbody>
                </tbody>
                    </table>

                    <script>
                        function loadStudents() {
                            fetch("students.json")
                                .then(response => response.json())
                                .then(data => {
                                    let tableBody = document.querySelector("#studentTable tbody");

                                    tableBody.innerHTML="";
                                    data.forEach(student => {
                                        let row = `
                                        <tr>
                                            <td>${student.id}</td>
                                            <td>${student.name}</td>
                                            <td>${student.department}</td>
                                            <td>${student.marks}</td>

                                        </tr>
                                        `;
                                        tableBody.innerHTML += row;
                                    });
                                })
                                .catch(error => {
                                    console.log('Error:', error)
                        });
                    }
                    </script>



                </body>
    </html>
```

students.json

```
    [
        {
            "id": 1,
            "name": "Rahul",
            "department": "CSE",
            "marks": 85
        },
        {
            "id": 2,
            "name": "Priya",
            "department": "AIML",
            "marks": 92
        },
        {
            "id": 3,
            "name": "Amit",
            "department": "Data Science",
            "marks": 88
        }
    ]
```

Working / Concepts Used

Concept / Function	Purpose
fetch("students.json")	Requests the JSON file.
response.json()	Parses the response body as JSON.
.then()	Processes the response and the parsed data.
forEach()	Iterates over each student object.
querySelector()	Selects the table body.
innerHTML	Clears and fills the table dynamically.
Template literals	Builds each table row using student properties.
.catch()	Logs an error if loading fails.

Output

- The page displays a “Student Details using Fetch()” heading and a Load Students button.
- Clicking Load Students reads the records from students.json.
- The student ID, name, department, and marks are displayed as table rows.
- The supplied JSON records include Rahul (CSE, 85), Priya (AIML, 92), and Amit (Data Science, 88).

Screenshot

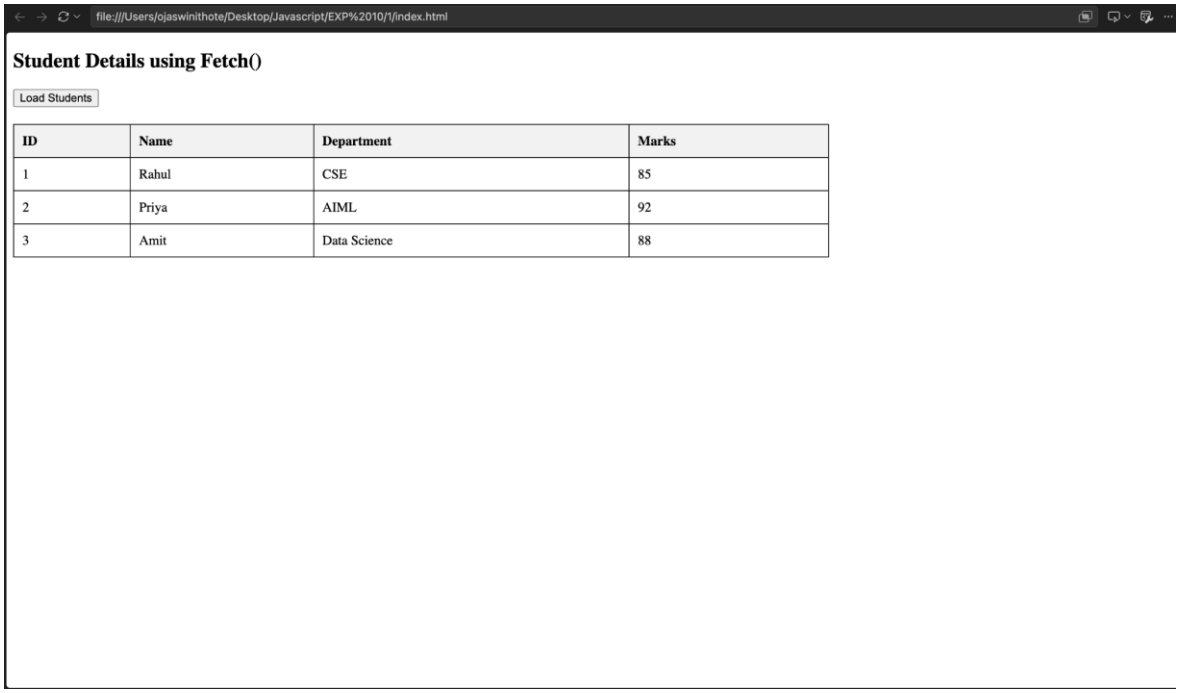


Figure 1: Practical Output — Student Details using fetch()

JSON Data

```
[
  {
    "id": 1,
    "name": "Rahul",
    "department": "CSE",
    "marks": 85
  },
  {
    "id": 2,
    "name": "Priya",
    "department": "AIML",
    "marks": 92
  },
  {
    "id": 3,
```

```
    "name": "Amit",
    "department": "Data Science",
    "marks": 88
  }
]
```

Practical: Student Details using JSON and jQuery

Aim

To create an interactive student management interface using jQuery $.getJSON() to load student records from a JSON file, add new student records dynamically, and render the records in a table while displaying the corresponding JSON representation.

Files Used

- index5.html — student entry form, records table, and JSON display
- script5.js — jQuery JSON loading and dynamic rendering logic
- student.json — initial student records

Program Code

index5.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Student Details using JSON</title>

    <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>

    <style>
        * {
            box-sizing: border-box;
            margin: 0;
            padding: 0;
        }

        body {
            font-family: Arial, sans-serif;
            background: #eef5f7;
            color: #263238;
            min-height: 100vh;
        }

        .container {
            width: 92%;
            max-width: 1150px;
            margin: 35px auto;
        }

        /* Header */

        .header {
            background: #ffffff;
            border-radius: 20px;
            padding: 28px 35px;
            margin-bottom: 25px;
            display: flex;
            align-items: center;
            justify-content: space-between;
            box-shadow: 0 8px 25px rgba(38, 50, 56, 0.08);
            border-left: 7px solid #2a9d8f;
        }
```

```
.header-text h1 {
    font-size: 30px;
    color: #214d57;
    margin-bottom: 8px;
}

.header-text p {
    color: #718096;
    font-size: 15px;
}

.json-icon {
    width: 75px;
    height: 75px;
    border-radius: 18px;
    background: #e0f4f1;
    color: #238276;
    display: flex;
    align-items: center;
    justify-content: center;
    font-size: 28px;
    font-weight: bold;
    border: 2px solid #bde4df;
}

/* Cards */

.card {
    background: #ffffff;
    padding: 27px;
    border-radius: 18px;
    margin-bottom: 25px;
    box-shadow: 0 8px 25px rgba(38, 50, 56, 0.07);
}

.card-title {
    display: flex;
    align-items: center;
    gap: 12px;
    margin-bottom: 22px;
}

.card-title span {
    width: 38px;
    height: 38px;
    border-radius: 10px;
    background: #fff0d9;
    color: #d9822b;
    display: flex;
    align-items: center;
    justify-content: center;
    font-weight: bold;
}

.card h2 {
    color: #294b53;
    font-size: 20px;
}

/* Form */

.form-grid {
    display: grid;
    grid-template-columns: repeat(2, 1fr);
    gap: 20px;
}

.form-group {
    display: flex;
    flex-direction: column;
```

```
}

label {
    font-size: 14px;
    font-weight: bold;
    color: #455a64;
    margin-bottom: 8px;
}

input {
    width: 100%;
    padding: 13px 14px;
    border: 1px solid #d6e0e3;
    border-radius: 9px;
    background: #f9fbfc;
    font-size: 15px;
    color: #263238;
    outline: none;
    transition: 0.2s;
}

input:focus {
    border-color: #2a9d8f;
    background: #ffffff;
    box-shadow: 0 0 0 3px rgba(42, 157, 143, 0.12);
}

/* Buttons */

.buttons {
    display: flex;
    gap: 14px;
    margin-top: 25px;
}

button {
    border: none;
    padding: 13px 24px;
    border-radius: 9px;
    font-size: 14px;
    font-weight: bold;
    cursor: pointer;
    transition: 0.2s;
}

#addButton {
    background: #2a9d8f;
    color: white;
}

#addButton:hover {
    background: #238276;
    transform: translateY(-1px);
}

#loadButton {
    background: #f4a261;
    color: white;
}

#loadButton:hover {
    background: #e58c47;
    transform: translateY(-1px);
}

#message {
    margin-top: 16px;
    padding: 10px 13px;
    border-radius: 7px;
    background: #edf8f6;
```

```
    color: #26776d;
    font-size: 14px;
    font-weight: bold;
}

/* Table */

.table-wrapper {
    overflow-x: auto;
}

table {
    width: 100%;
    border-collapse: collapse;
    overflow: hidden;
    border-radius: 12px;
}

thead {
    background: #214d57;
    color: white;
}

th {
    padding: 15px;
    text-align: left;
    font-size: 14px;
}

td {
    padding: 14px 15px;
    border-bottom: 1px solid #edf1f2;
    font-size: 14px;
}

tbody tr:nth-child(even) {
    background: #f7fafb;
}

tbody tr:hover {
    background: #eaf6f4;
}

td:first-child {
    font-weight: bold;
    color: #2a9d8f;
}

td:last-child {
    font-weight: bold;
    color: #e07a3f;
}

/* JSON */

.json-box {
    background: #202b33;
    color: #b9e4dc;
    padding: 22px;
    border-radius: 12px;
    overflow-x: auto;
    white-space: pre-wrap;
    font-family: Consolas, monospace;
    font-size: 14px;
    line-height: 1.7;
    min-height: 120px;
    border-left: 5px solid #2a9d8f;
}
```

```
        /* Footer */

        .footer-note {
            text-align: center;
            color: #84949a;
            font-size: 13px;
            margin-top: 5px;
            margin-bottom: 20px;
        }

        /* Responsive */

        @media (max-width: 700px) {

            .container {
                width: 94%;
                margin: 20px auto;
            }

            .header {
                padding: 22px;
            }

            .header-text h1 {
                font-size: 24px;
            }

            .json-icon {
                width: 58px;
                height: 58px;
                font-size: 22px;
            }

            .form-grid {
                grid-template-columns: 1fr;
            }

            .buttons {
                flex-direction: column;
            }

            button {
                width: 100%;
            }

            .card {
                padding: 20px;
            }
        }
    </style>
</head>

<body>

    <div class="container">

        <div class="header">

            <div class="header-text">
                <h1>Student Details using JSON</h1>
                <p>Manage student information using JavaScript, JSON and jQuery</p>
            </div>

            <div class="json-icon">
                { }
            </div>
```

```
</div>


<div class="card">

    <div class="card-title">
        <span>01</span>
        <h2>Enter Student Details</h2>
    </div>

    <div class="form-grid">

        <div class="form-group">
            <label for="studentId">Student ID</label>
            <input
                type="number"
                id="studentId"
                placeholder="Enter student ID">
        </div>

        <div class="form-group">
            <label for="studentName">Student Name</label>
            <input
                type="text"
                id="studentName"
                placeholder="Enter student name">
        </div>

        <div class="form-group">
            <label for="department">Department</label>
            <input
                type="text"
                id="department"
                placeholder="Enter department">
        </div>

        <div class="form-group">
            <label for="marks">Marks</label>
            <input
                type="number"
                id="marks"
                placeholder="Enter marks">
        </div>

    </div>

    <div class="buttons">
        <button id="addButton">Add Student</button>
        <button id="loadButton">Load Students</button>
    </div>

    <p id="message">Enter student details to begin.</p>

</div>



<div class="card">

    <div class="card-title">
        <span>02</span>
        <h2>Student Records</h2>
    </div>

    <div class="table-wrapper">

        <table>
```

```
                    <thead>
                        <tr>
                            <th>Student ID</th>
                            <th>Name</th>
                            <th>Department</th>
                            <th>Marks</th>
                        </tr>
                    </thead>

                    <tbody id="studentTableBody"></tbody>

                </table>

            </div>

        </div>



        <div class="card">

            <div class="card-title">
                <span>03</span>
                <h2>JSON Representation</h2>
            </div>

            <div class="json-box" id="jsonData">
                No student data available.
            </div>

        </div>

        <p class="footer-note">
            Student Management • JSON • jQuery
        </p>

    </div>

    <script src="script5.js"></script>

</body>
</html>
```

script5.js

```
const studentId = document.getElementById("studentId");
const studentName = document.getElementById("studentName");
const department = document.getElementById("department");
const marks = document.getElementById("marks");


const addButton = document.getElementById("addButton");
const loadButton = document.getElementById("loadButton");


const tableBody = document.getElementById("studentTableBody");
const jsonData = document.getElementById("jsonData");
const message = document.getElementById("message");


let students = [];
let sessionStudents = [];


addButton.addEventListener("click", function () {

    if (
        studentId.value === "" ||
        studentName.value === "" ||
        department.value === "" ||
        marks.value === ""
    ) {
```

```
        alert("Please enter all student details.");
        return;
    }

    const student = {
        id: Number(studentId.value),
        name: studentName.value,
        department: department.value,
        marks: Number(marks.value)
    };

    sessionStudents.push(student);
    students.push(student);

    displayStudents();

    const studentJSON = JSON.stringify(students, null, 4);

    jsonData.textContent = studentJSON;

    message.textContent = "Student added successfully.";

    studentId.value = "";
    studentName.value = "";
    department.value = "";
    marks.value = "";
});

loadButton.addEventListener("click", function () {

    $.getJSON("student.json")
        .done(function (data) {

            students = data.concat(sessionStudents);

            displayStudents();

            const studentJSON = JSON.stringify(students, null, 4);

            jsonData.textContent = studentJSON;

            message.textContent =
                "Student records loaded successfully.";

        })
        .fail(function () {

            message.textContent =
                "Error loading student.json.";

        });
});

function displayStudents() {

    tableBody.innerHTML = "";

    students.forEach(function (student) {

        const row = document.createElement("tr");

        row.innerHTML = `
            <td>${student.id}</td>
            <td>${student.name}</td>
            <td>${student.department}</td>
```

```
            <td>${student.marks}</td>
        `;

        tableBody.appendChild(row);
    });
}
```

student.json

```
[
    {
        "id": 101,
        "name": "Aarav Sharma",
        "department": "Computer Science",
        "marks": 85
    },
    {
        "id": 102,
        "name": "Priya Patil",
        "department": "Information Technology",
        "marks": 91
    },
    {
        "id": 103,
        "name": "Rohan Mehta",
        "department": "Electronics",
        "marks": 78
    }
]
```

Features

- Enter student ID, name, department, and marks.
- Add a student to the current page session.
- Load initial student records from student.json using $.getJSON().
- Render records dynamically in the Student Records table.
- Display the current student array as formatted JSON using JSON.stringify().
- Show a success message after loading or adding records, and an error message if JSON loading fails.

JavaScript / jQuery Concepts Used

Concept / Function	Purpose
$.getJSON()	Loads and parses student.json as JSON.
.done()	Runs when the JSON file loads successfully.
.fail()	Handles a failure while loading the JSON file.
addEventListener()	Responds to Add Student and Load Students button clicks.
Array.push()	Adds new records to the in-memory arrays.
Array.concat()	Combines initial JSON records with records added during the current session.
displayStudents()	Clears and rebuilds the student table.
document.createElement()	Creates each table row dynamically.
appendChild()	Adds each generated row to the table body.
JSON.stringify()	Converts the student array into formatted JSON text.
textContent	Displays JSON and status messages.

Output

- The page shows a form for student ID, name, department, and marks.
- Load Students retrieves the initial records from student.json using $.getJSON().
- Add Student validates that all fields are entered, then adds the record to the current page session.
- The table updates dynamically to display student records.
- The JSON Representation section shows the current array as formatted JSON.

- Newly added records are held in JavaScript memory for the current page session; the code does not write them back into the JSON file.

Screenshots

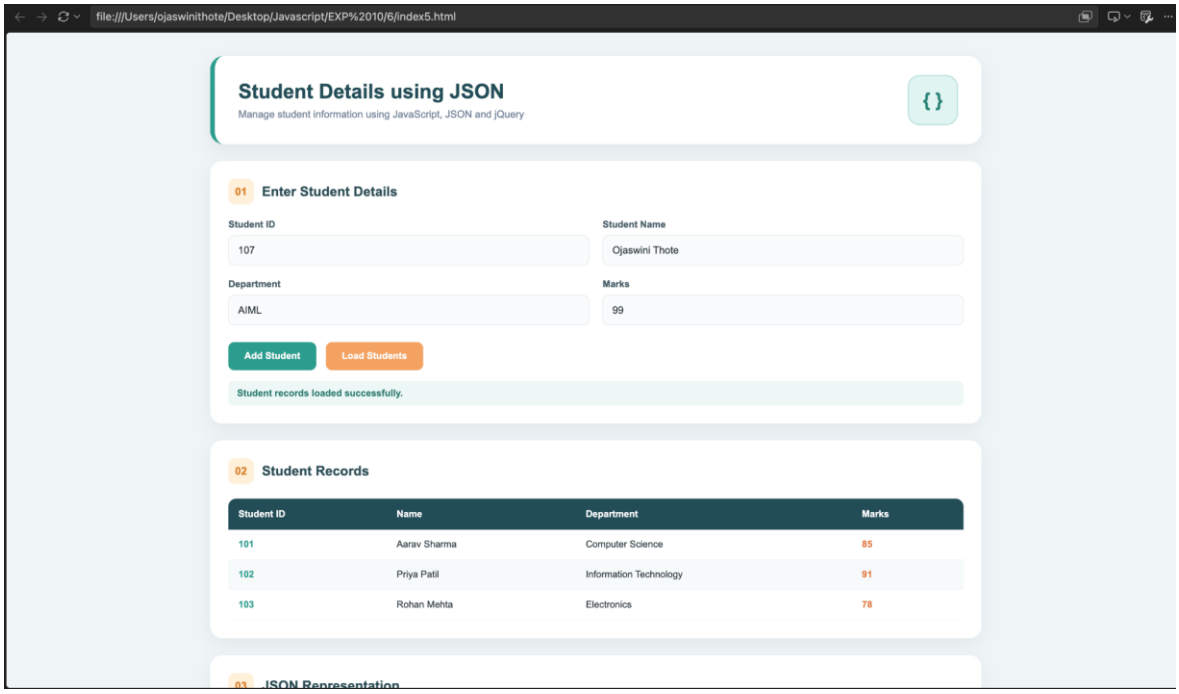


Figure 2: Case Study Output — Student Entry Form and Records

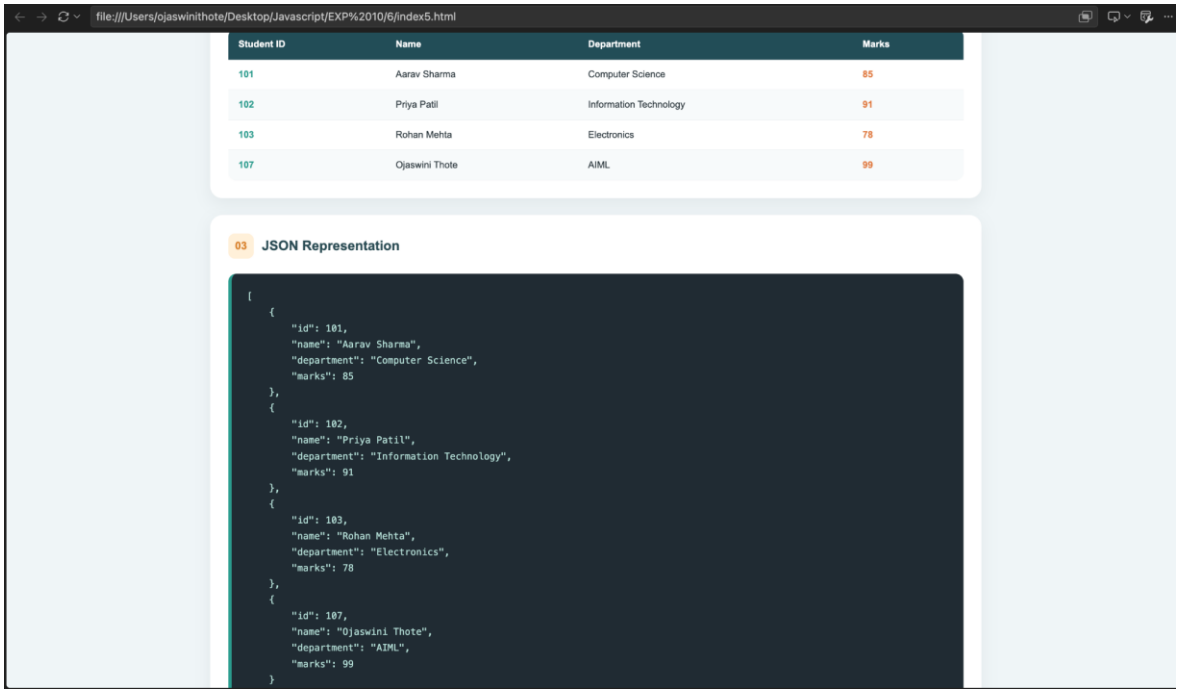


Figure 3: Case Study Output — Student Records and JSON Representation

Result / Conclusion

The practical successfully demonstrated loading JSON data using fetch() and displaying student records dynamically in an HTML table. The case study demonstrated loading JSON data with jQuery $.getJSON(), adding student records during the

current page session, dynamically updating the table, and presenting the student array as formatted JSON. The experiment illustrates asynchronous JSON loading, JavaScript array operations, DOM manipulation, and dynamic table rendering.